3

PART 1: FOUNDATION

# Specifying what the system must do

How do you turn a vision into requirements an AI assistant can't misread?

The Riverside Clinics vision from the last chapter settles why the system exists: some patients go without care rather than state the reason for a visit where staff and other patients can hear it, and the clinic wants booking to be private. Set against the one-line request that started this book, that is a large step forward. A developer picks it up to start building the booking flow, and inside the first hour she runs out of things the vision tells her.

What happens if a patient tries to book at three in the morning? The vision hasn't said. How far into the future can someone book, next week, next year, ever? It hasn't said that either. What happens when two patients grab the same ten o'clock slot half a second apart? Nothing in the vision answers that. The vision explained why the system exists. It never promised to explain what the system does once it's running.

**CHAPTER PROMISE**

By the end of this chapter, you can pin one piece of system behavior down precisely enough that an AI assistant tests it instead of guessing it, and turn a whole booking interaction into a single, step-by-step description complete enough to implement directly.

Reading time: 12 minutes

# Where the unanswered questions go instead of into a document

Nobody wrote the three questions from the opening scene into a document, but someone still has to answer them before the booking form works. Left unstated, the AI answers them the moment it writes the reservation logic, and it answers with a plausible default instead of a decision. It might let patients book a year out, because nothing told it ninety days was the limit doctors' schedules support. It might let two patients reserve the same slot in the gap between checking availability and confirming it, because nothing told it that gap mattered enough to close.

The code runs. It passes a demo. None of that tells you whether the horizon it picked, or the race condition it left open, matches what Riverside's clinics can actually support.

A patient books an appointment fourteen months out. The clinic has no idea whether that doctor will still be practicing there by then. Nobody notices until the appointment is already confirmed and the patient has already taken the afternoon off work. That failure traces back to one line nobody wrote down: how far in advance is a booking allowed to reach.

Someone still has to answer questions like that before code does, and writing the answer down doesn't invent new work. It moves a decision that was always going to get made, out of the AI's guess and into something a human actually chose.

# The unit AI implements best

A **requirement** is a statement about system behavior you can observe and test, and it says nothing about how the code achieves it. "The system should be user-friendly" fails that test twice over: nobody can watch the system and confirm it, and nobody can write a test case that passes or fails against a feeling. "A patient can complete a booking in three minutes or less, in five clicks or fewer" passes both. You can time it. You can count the clicks. You can tell, without asking anyone's opinion, whether the system meets it.

A single requirement describes one fact in isolation, though. "The system displays available appointment slots." "The system validates the patient's email address." "The system sends a confirmation within one minute." Each one is testable on its own, and none of them says how the facts fit together, or in what order, or what happens when one of them fails partway through. A patient's work rarely arrives as an isolated fact. It arrives as a trip from "I need an appointment" to "I have one," and that whole path is what a requirement list was never built to describe.

A **use case** describes that whole path: one actor, pursuing one goal, from start to finish, as a sequence of steps a system and a person carry out together. "Patient books an appointment" is a use case. It contains most of the requirements above, in the order they actually happen, plus the moments where something can go wrong along the way.

> **A requirement tells an AI assistant one fact about the system. A use case tells it the whole interaction that fact belongs to. Handing over the whole interaction is central to Simon Martinelli's methodology, which this book draws on throughout. An assistant that can see how the steps relate has less left to assemble on its own.**

# Turning an open question into a written rule

Requirements rarely arrive fully formed. They surface in conversation, usually between someone who understands the users, someone who understands the business constraints, and someone who understands the rule that's been sitting in a colleague's head for years without ever reaching a document. Here is that kind of conversation, condensed, negotiating the booking horizon the opening scene left open.

### A workshop, condensed

**Facilitator:** What has to be true before a patient can book?

**Domain expert:** They need to see when the doctor is free.

**Product owner:** Only during the hours we're open. We close at five.

**Business analyst:** We actually want booking available around the clock, any day, for future dates. Just not too far out.

**Domain expert:** How far is too far?

**Business analyst:** Doctors' schedules are only confirmed ninety days ahead. Past that, nobody knows who's working.

**Facilitator:** So the rule is: patients can book any time, any day, for slots inside clinic hours, never more than ninety days out.

Illustrative, built to show how a workshop narrows a vague rule to a specific number. The shape of the conversation is typical of how this kind of decision gets made; the words are not a transcript of an actual meeting.

1. Ask what has to be true before the action happens, and write down the first answer, even when it's vague.
2. Push on every edge you can think of: after hours, a year from now, two people acting at the same instant. Most vague answers survive the first push and fail the second.
3. Stop when someone names an actual number or limit. Vague phrases like "not too far out" are a sign to keep pushing until a specific figure, like "ninety days," replaces them.
4. Write the rule as one sentence a developer, or an AI assistant, could test against, and keep it where both can find it.

# The Schedule Appointment use case, start to finish

Filled in below, including the ninety-day rule the workshop just settled.

### Use case: Schedule Appointment

**Actor:** Clinic patient, or a receptionist booking on the patient's behalf.

**Preconditions:**

- Patient, or a receptionist on the patient's behalf, is registered and logged in.
- A doctor has open availability within clinic hours (8 a.m.–5 p.m., Mon–Fri).

**Main success scenario:**

1. Patient selects a doctor.
2. System displays that doctor's open slots, fifteen-minute intervals.
3. Patient selects a time.
4. System re-checks that the slot is open, then reserves it.
5. Patient confirms the booking.
6. System emails confirmation: doctor, time, location, cancellation link.
7. System shows a booking summary; the appointment appears on the patient's dashboard and the doctor's schedule.

**Alternative flows:**

- **No slots available, step 2:** offer a different doctor or a later date range.
- **Slot taken before reservation, step 4:** notify the patient, discard the attempt, redisplay availability.
- **Patient cancels, any step:** discard the reservation, return to doctor selection.

**Postconditions:**

- Appointment exists, linked to patient and doctor; confirmation email sent.
- Doctor's schedule and patient's dashboard both reflect the appointment.

**Business rules:**

- No booking more than 90 days ahead, the confirmed horizon for doctors' schedules; slots run fifteen minutes, within clinic hours (8 a.m.–5 p.m., Mon–Fri).
- Cancellation requires at least 24 hours' notice; otherwise, call the clinic.
- Doctor availability is entered by clinic staff, not pulled from an external calendar.

# Where a use case can look finished and still mislead

**▲ WARNING**

A use case can name every step and still leave the work half done. If a step says "the system sends a confirmation email" and never says what the email contains, the AI assistant has to invent a subject line, a greeting, and a layout, and it will. If one use case calls the actor "Doctor" and another calls the same person "Physician," the AI assistant has no way to know they're the same thing. Before any use case goes to an AI assistant, check that every step is filled in and that its terms match every other use case in the specification.

| | |
|---|---|
| **Handing an AI assistant a backlog of user stories instead of a use case** | "As a patient, I want to book an appointment online, so I don't have to call" plans the work well and specifies almost nothing: it doesn't say what happens with no slots free, or two patients racing for one. A single story like this one usually bundles several use cases together, and an AI assistant asked to implement it has to split that bundling apart itself. |
| **Writing use cases for a roadmap conversation** | A use case's preconditions and alternative flows are exactly the detail a planning meeting doesn't need and a backlog groom shouldn't have to sit through. User stories exist for that conversation. Use cases exist for the one an AI assistant needs before it writes anything. |

# Field guide: the use-case template

This is the shape Martinelli's methodology recommends: title, actor, preconditions, main scenario, alternative flows, postconditions, business rules. The template below is that shape with the blanks named.

- ☐ Title: one goal, usually a verb phrase, one actor accomplishing it.
- ☐ Actor: who performs this use case, named specifically, not "the user."
- ☐ Preconditions: everything that must already be true before step one can run.
- ☐ Main success scenario: the complete happy path, numbered, one action per step.
- ☐ Alternative flows: every place the main scenario can go wrong, each tied to the step where it branches.
- ☐ Postconditions: everything that must be true once the use case has succeeded.
- ☐ Business rules: the constraints that apply across the whole use case, each written as a testable limit.

Fill in every line before a use case goes to an AI assistant. A blank line here is a decision the AI assistant will make for you.

# What to remember

- A requirement is testable, observable, and silent on implementation. A sentence nobody can test is only an opinion.
- A vision leaves decisions open, like booking horizons and race conditions between two people acting at once. A workshop that pushes past the first answer is what turns an open decision into a written rule.
- A use case describes one complete interaction rather than an isolated fact, and that completeness is why this book treats a use case, not a requirement list, as the unit to hand an AI assistant.
- User stories plan work; use cases drive implementation. Use one where the other belongs and you get a backlog nobody can build from, or a specification nobody can prioritize against.

### QUESTIONS FOR YOUR TEAM

- How many requirements has your team written this quarter that name an actual number, against how many only describe a feeling, like "fast" or "easy"?
- If you handed an AI assistant your last three user stories with nothing else, what would it have had to guess?

### ONE ACTION TO TAKE NEXT

Take the next use case your team plans to hand to an AI assistant and run it against this chapter's field guide before it goes anywhere. Count the lines you cannot fill in; each one is a question for someone outside your team.

---

TRANSITION. Every step in the Schedule Appointment use case talks about patients, doctors, and appointments as though their shape is already agreed. The next chapter settles that agreement: a domain model precise enough that an AI assistant stops inventing what a doctor record contains.

Specifying what the system must do ends here.